

A Problem in gosec Taint Analysis

"Wait... filepath.Clean() doesn't do that!"

Nicholas Capo
Staff Infrastructure Engineer
Axios Media

gosec

- “Inspects source code for security problems by scanning the Go AST and SSA code representation.”
- Available standalone or as part of `golangci-lint`
- Finds things like:
 - G303 — Creating tempfile using a predictable path (AST)
 - G403 — Ensure minimum RSA key length of 2048 bits (AST)
- Added around 2026-02:
 - G7xx: taint analysis rules (SQL injection, command injection, path traversal, SSRF, XSS, log, SMTP injection, SSTI, unsafe deserialization, and open redirect)
 - Specifically:
 - G703 — Path traversal via taint analysis (Taint)

SSA

- Static Single-Assignment form
- <https://go.dev/src/cmd/compile/internal/ssa/README>

Go		SSA
<pre>y := 1 y := 2 x := y</pre>	→	<pre>y1 := 1 y2 := 2 x1 := y2</pre>

Optimizations

https://go.dev/src/cmd/compile/internal/ssa/_gen/generic.rules

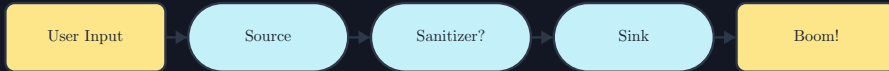
```
(Not (Neg(64|32|16|8|B|Ptr|64F|32F) x y)) => (Eq(64|32|16|8|B|Ptr|64F|32F) x y)
(Not (Less(64|32|16|8) x y)) => (Leq(64|32|16|8) y x)
```

Details

GOSSAFUNC=Example go build

Taint Analysis

- Analysis takes place at the SSA level
- Uses `golang.org/x/tools/go/analysis.Analyzer`



Sources and Sinks

gosec/analyzers/pathtraversal.go:25

```
return taint.Config{
    Sources: []taint.Source{
        {Package: "os", Name: "Args", IsFunc: true},
        {Package: "os", Name: "Getenv", IsFunc: true},
    },
    Sinks: []taint.Sink{
        {Package: "os", Method: "Open"},
        {Package: "os", Method: "OpenFile"},
        ...
    },
}
```

Examples

Oneliner

```
f, err := os.Open(os.Getenv("EXAMPLE_PATH"))
```

A Little More Complex

```
pwd, err := os.Getwd()  
filename = filepath.Join(pwd, "prefix", os.Args[1])  
i, err = os.Stat(filename)
```

Sanitizers

```
taint.Config{
  Sanitizers: []taint.Sanitizer{
    // filepath.Clean normalizes and removes traversal components
    {Package: "path/filepath", Method: "Clean"},
    // filepath.Abs calls Clean internally (per Go docs)
    {Package: "path/filepath", Method: "Abs"},
    ...
    // url.PathEscape escapes path components
    {Package: "net/url", Method: "PathEscape"},

    // Integer conversions eliminate path traversal vectors entirely –
    // the result can never contain "/" or ".." characters.
    {Package: "strconv", Method: "Atoi"},
    ...
  },
}
```

But!

filepath.Clean() doesn't do that!

From pkg.go.dev:

Clean returns the shortest path name equivalent to path by purely lexical processing. It applies the following rules iteratively until no further processing can be done:

[...]

3. Eliminate each inner .. path name element (the parent directory) along with the non-.. element that precedes it.
4. Eliminate .. elements that begin a rooted path: that is, replace /. . by / at the beginning of a path, assuming Separator is /.

Meaning it doesn't so much clean as it does evaluate!

A Simple Test

src/simple/main.go

```
package main

import (
    "fmt"
    "path/filepath"
)

func main() {
    untrusted := "../.../etc/passwd"

    fmt.Println(filepath.Clean(untrusted))
    fmt.Println("/path/example" + filepath.Clean(untrusted))
    fmt.Println(filepath.Clean("/path/example/" + untrusted))
    fmt.Println(filepath.Clean(filepath.Join("/", "path", "example", untrusted)))
}
```

Output

```
$ go run ./src/simple
/etc/passwd
/path/example/etc/passwd
/etc/passwd
/etc/passwd
```

But does **gosec** find these?

src/bad/main.go

```
package main

import (
    "fmt"
    "os"
    "path/filepath"
)

func main() {
    pwd, _ := os.Getwd()
    filename := filepath.Join(pwd, "prefix", os.Args[1])

    _, err := os.Stat(filename)
    fmt.Printf("stat filename=%s error=%v\n", filename, err)
}
```

Bad Output

```
-----  
$ go run ./src/bad file  
stat filename=prefix/file error=<nil>
```

```
-----  
$ go run ./src/bad ../../../../../../../../../../../../../../etc/passwd  
stat filename=/etc/passwd error=<nil>
```

```
-----  
$ go run github.com/securego/gosec/v2/cmd/gosec@v2.28.0 --color=false --include G703 ./src/bad  
Results:
```

```
[src/bad/main.go:13] - G703 (CWE-22): Path traversal via taint analysis (Confidence: HIGH, Severity: HIGH)  
12:  
> 13:   _, err := os.Stat(filename)  
14:   fmt.Printf("stat filename=%s error=%v\n", filename, err)
```

Autofix:

Summary:

```
Gosec   : dev  
Files   : 1  
Lines   : 15  
Nosec   : 0  
Issues  : 1
```

src/good/main.go

```
package main

import (
    "fmt"
    "net/url"
    "os"
    "path/filepath"
)

func main() {
    pwd, _ := os.Getwd()
    path := filepath.Join(pwd, "prefix", os.Args[1])

    //gosec says: // filepath.Clean normalizes and removes traversal components
    filename := filepath.Clean(path)
    _, err := os.Stat(filename)
    fmt.Printf("Clean filename=%s error=%v\n", filename, err)

    //gosec says: // filepath.Abs calls Clean internally (per Go docs)
    filename, _ = filepath.Abs(path)
    _, err = os.Stat(filename)
    fmt.Printf("Abs filename=%s error=%v\n", filename, err)

    //gosec says: // url.PathEscape escapes path components
    filename, _ = url.PathUnescape(url.PathEscape(path))
    _, err = os.Stat(filename)
    fmt.Printf("PathEscape filename=%s error=%v\n", filename, err)
}
```

“Good” Output

```
-----  
$ go run ./src/good file  
Clean filename=prefix/file error=<nil>  
Abs filename=prefix/file error=<nil>  
PathEscape filename=prefix/file error=<nil>
```

```
-----  
$ go run ./src/good ../../../../../../../../../../../../../../etc/passwd  
Clean filename=/etc/passwd error=<nil>  
Abs filename=/etc/passwd error=<nil>  
PathEscape filename=/etc/passwd error=<nil>
```

```
-----  
$ go run github.com/securego/gosec/v2/cmd/gosec@v2.28.0 --color=false --include G703 ./src/good  
Results:
```

Summary:

```
Gosec   : dev  
Files   : 1  
Lines   : 28  
Nosec   : 0  
Issues  : 0
```

Uh Oh!

The Solution(?)

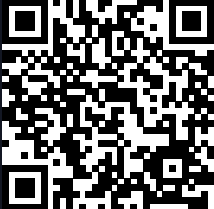
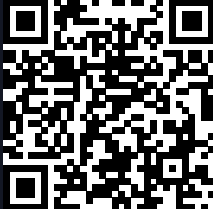
- Open an issue in `gosec`
- Probably need to change the sanitizers to use `os.Root`

From `pkg.go.dev`:

`Root` may be used to only access files within a single directory tree.

Methods on `Root` can only access files and directories beneath a root directory. If any component of a file name passed to a method of `Root` references a location outside the root, the method returns an error. File names may reference the directory itself (`.`).

Links

Slides and Source Code	Gosec	Rules	<code>pathtraversal.go:25</code>
			
SSA: Go Blog	SSA: Wikipedia	SSA: Rules	<code>os.Root</code>
			